JV32 Performance Analysis

Table of Contents

1. Part I: Architecture for Performance . . . 2
1.1. Pipeline Overview . . . 2
1.2. Branch Predictor (`BP_EN`) . . . 3
1.2.1. Motivation . . . 3
1.2.2. Implementation . . . 3
1.2.3. Performance Impact . . . 4
1.3. Register Forwarding . . . 4
1.3.1. Motivation . . . 4
1.3.2. Implementation . . . 4
1.3.3. Performance Impact . . . 4
1.3.4. Load-Use Stall (Residual Hazard) . . . 5
1.4. Fast Arithmetic Units . . . 5
1.4.1. Multiplier (`FAST_MUL`, `MUL_MC`) . . . 5
1.4.2. Divider (`FAST_DIV`) . . . 5
1.4.3. Barrel Shifter (`FAST_SHIFT`) . . . 6
1.4.4. Performance Impact Summary . . . 6
1.5. RVC Instruction Decompressor (`jv32_rvc`) . . . 6
1.5.1. Motivation . . . 6
1.5.2. Implementation . . . 6
1.5.3. Performance Impact . . . 7
1.6. Store Buffer . . . 7
1.7. Simulation Environment . . . 7
1.8. Maximum Performance Results . . . 8
1.8.1. CoreMark . . . 8
1.8.2. Dhrystone . . . 8
1.9. Branch Predictor Impact Analysis . . . 9
1.9.1. Results Table . . . 9
1.9.2. Analysis . . . 9
1.9.3. Misprediction Rate Comparison . . . 10
1.10. Compressed Instruction (RVC) Impact Analysis . . . 10
1.10.1. Results Table . . . 10
1.10.2. Analysis . . . 11

1.11. CPI Statistics from Trace Analysis . . . 11
1.11.1. CPI Decomposition . . . 11
1.11.2. CoreMark Instruction Mix . . . 12
1.11.3. Dhrystone Instruction Mix . . . 12
1.11.4. Stall Cycle Distribution . . . 13
1.11.5. Observations . . . 14
1.12. Summary Table . . . 14

1. Part I: Architecture for Performance

This section describes the microarchitectural features of the JV32 3-stage in-order pipeline that are directly responsible for performance. Each feature is paired with the parameter that controls it and a summary of the performance impact it provides.

1.1. Pipeline Overview

JV32 implements a 3-stage in-order pipeline: **IF** → **EX** → **WB**.

Stage	Function
IF	PC register, instruction fetch request via the RVC expander (`jv32_rvc`)
EX	Decode, register file read, ALU, address calculation, CSR, hazard detect
WB	Memory response, register writeback, redirect, exception

The compact three-stage design keeps the branch misprediction penalty to a single flush cycle and minimises the register-file-read-to-ALU latency. The trade-off is that every load instruction is directly followed by a one-cycle stall when the very next instruction consumes the loaded value (load-use hazard).

The ideal CPI (no stalls, no hazards) is 1.0. The sources of overhead in practice are:

- **Load-use stalls** — the loaded value is not yet written back when the dependent instruction reaches EX. This stall is inherent to a 3-stage pipeline and cannot be eliminated without a deeper pipeline or a store/load buffer.
- **Control hazards** — taken branches, JAL pre-decode misses, JALR redirects, and branch mispredictions each cost one pipeline flush cycle.
- **Multi-cycle arithmetic** — serial multiply/divide/shift (when FAST_* parameters are disabled) stall the entire pipeline until the operation completes.

1.2. Branch Predictor (BP_EN)

1.2.1. Motivation

In a 3-stage pipeline, a taken branch that is detected in the EX stage has already caused the wrong-path instruction to enter the IF→EX register. That instruction must be squashed (one bubble inserted), costing one cycle. For a workload with ~53 % taken-branch rate the overhead without any prediction is significant.

1.2.2. Implementation

JV32 implements a lightweight **static-plus-dynamic** predictor composed of three layers that run in parallel at the RVC output (before the IF/EX register is written):

Layer	Type	Description
BTFNT	Static	**Backward-Taken** / **Forward-Not-Taken**: if the branch immediate is negative (sign bit = 1 in `instr[31]`) the branch targets a lower address (a loop back-edge) and is predicted taken; otherwise not-taken. This rule is correct for the majority of loop-back branches without any state.
JAL pre-decode	Static unconditional	JAL encodes the full target in the instruction word. The front-end pre-decodes JAL and redirects `pc_if` to the target in the same cycle the instruction is presented by the RVC, achieving **zero-penalty** JAL execution.
L0 last-branch	1-entry dynamic	A single entry that records the PC and target of the most recently **resolved** conditional branch plus whether it was actually taken. On the next fetch of the same PC the L0 entry takes priority over BTFNT, allowing a forward-taken branch that was taken last time to be predicted taken again. This catches recurring loop heads whose direction cannot be inferred from the sign bit alone.

The predictor operates entirely in the IF stage, redirecting `pc_if` and recording `bp_taken=1` in the IF/EX pipeline register. In EX, the actual branch outcome is compared against the prediction:

- **Correct prediction** (taken and predicted taken, or not-taken and not predicted): no redirect — zero-cycle penalty.
- **Misprediction** (wrong direction): one-cycle flush.
- **JALR** (target unknown at IF time): always one-cycle EX redirect.

1.2.3. Performance Impact

The branch predictor reduces the per-branch redirect rate from 52–53 % (all taken branches) to 8.8–21.6 % (only mispredictions). The result is a measurable CPI improvement shown in [Part II].

1.3. Register Forwarding

1.3.1. Motivation

In a 3-stage pipeline the write-back result of instruction *N* is available at the end of WB, one cycle after *N* was in EX. Without forwarding, instruction *N+1* that reads the same register would have to stall two cycles waiting for the value to propagate through WB back to the register file and be re-read in EX.

1.3.2. Implementation

JV32 implements a single WB→EX forwarding path. Every cycle the WB stage checks whether its write-back destination (`ex_wb_r.rd_addr`) matches either source register (`dec_rs1`, `dec_rs2`) of the EX-stage instruction. If a match is found the forwarded result is selected instead of the register-file output:

```
always_comb begin
    fwd_rs1 = rs1_data;
    fwd_rs2 = rs2_data;
    if (ex_wb_r.valid && ex_wb_r.reg_we && ex_wb_r.rd_addr != 5'd0
        && (!load_uses_sram || dmem_resp_valid)) begin
        if (ex_wb_r.rd_addr == dec_rs1) fwd_rs1 = wb_rd_data;
        if (ex_wb_r.rd_addr == dec_rs2) fwd_rs2 = wb_rd_data;
    end
end
```

For non-load instructions (`wb_rd_data = ex_wb_r.rd_data`) the result is a registered flip-flop output and is unconditionally stable, so forwarding is always safe. For load instructions the result is the combinatorial SRAM output (`dmem_resp_data`) and forwarding is enabled only when `dmem_resp_valid=1`, i.e. when the SRAM has provided a valid response.

1.3.3. Performance Impact

Forwarding eliminates all RAW (read-after-write) stalls **except** the load-use case. Without forwarding, every instruction following a non-load register producer would stall two cycles. For the CoreMark workload this would add approximately 0.24 CPI overhead (24 % of instructions produce a result consumed by the next instruction); with forwarding only load-use stalls remain (~0.060 CPI).

1.3.4. Load-Use Stall (Residual Hazard)

When a load instruction is immediately followed by an instruction that consumes the loaded register, the dependent instruction is stalled for exactly one cycle in EX while the load completes its WB memory response:

```
Cycle N  : LOAD in EX — issues dmem_req_addr
Cycle N+1: LOAD in WB — dmem_resp_valid=1, wb_rd_data available
           Dependent in EX — STALL (load_use_stall=1)
           Forwarded wb_rd_data used by dependent in EX
Cycle N+2: Dependent in WB — retires
```

This is an inherent 3-stage pipeline constraint. A **store buffer** (write queue) would not eliminate this hazard; only a 4-stage pipeline (separating address generation from data forwarding) or a value predictor could do so.

1.4. Fast Arithmetic Units

The pipeline stalls the entire pipeline while a multi-cycle arithmetic unit is in use. Three independent parameters control whether each unit is single-cycle or multi-cycle.

1.4.1. Multiplier (`FAST_MUL`, `MUL_MC`)

FAST_MUL	MUL_MC	Latency	Description
0	—	Variable (1–32 cycles)	Serial shift-and-add. Uses minimal flip-flop area but stalls the pipeline for each active bit.
1	1	2 cycles	Two-stage pipelined multiplier. Stage 1 computes four unsigned 16×16 partial products; stage 2 accumulates and sign-corrects. Better timing closure than the 1-cycle variant.
1	0	1 cycle	Fully combinatorial 32×32 multiplier. No pipeline stall; `ready` is permanently asserted. Maximum MUL throughput.

For maximum performance this document uses `FAST_MUL=1 MUL_MC=0`.

1.4.2. Divider (`FAST_DIV`)

FAST_DIV	Latency	Description
0	Variable (up to 32 cycles)	Serial restoring divider. Very low area. Stalls the pipeline for the full quotient width.
1	1 cycle	Fully combinatorial restoring divider. Eliminates all DIV stall cycles. Suitable when maximum performance is required and the timing constraint allows the combinatorial depth.

1.4.3. Barrel Shifter (FAST_SHIFT)

FAST_SHIFT	Latency	Description
0	Variable (0–31 cycles)	1-bit-per-cycle serial shifter. Minimum area but stalls for shift amounts > 0.
1	1 cycle	Full-width barrel shifter (SLL/SRL/SRA all in one cycle). No pipeline stall for any shift amount.

1.4.4. Performance Impact Summary

For a workload with 3.7 % MUL/DIV instructions (CoreMark) switching from fully serial arithmetic (`FAST_MUL=0 FAST_DIV=0 FAST_SHIFT=0`) to fully fast arithmetic can reduce arithmetic-stall overhead by tens of cycles per MUL and hundreds per DIV. For Dhrystone (0.6 % MUL/DIV) the impact is smaller. Both benchmarks in this document use the maximum-performance configuration (`FAST_MUL=1 MUL_MC=0 FAST_DIV=1 FAST_SHIFT=1`).

1.5. RVC Instruction Decompressor (jv32_rvc)

1.5.1. Motivation

The RISC-V C extension (RVC) provides 16-bit encodings for the most common 32-bit instructions. Executing the same workload with compressed instructions reduces code size by ~20–30 %, which improves utilisation of instruction-fetch bandwidth and benefits real systems with an instruction cache. The on-chip TCM used in simulation does not cache, so the code-density advantage is absent in simulation; what the RVC expander does cost is additional latency in the IF stage.

1.5.2. Implementation

`jv32_rvc` sits between the instruction memory response and the IF/EX pipeline register. It

maintains a one-instruction buffer to handle the half-word alignment case where a 32-bit instruction straddles a 4-byte boundary. The expander converts any 16-bit `C. instruction to its 32-bit I/M equivalent *before` the branch predictor and decoder see the instruction, so all downstream logic remains ISA-width agnostic.

The cost is: * One extra cycle of fetch latency when a 32-bit instruction crosses a 4-byte boundary (one "alignment bubble"). * Slightly more complex `rvc_mem_ready` gating logic that can occasionally prevent the PC from advancing.

1.5.3. Performance Impact

In the simulation environment with single-cycle TCM the RVC path adds a small overhead compared to uncompressed code (`rv32ima_zicsr`), as shown in [Part II]. In a real system with instruction cache the picture reverses: smaller code means fewer cache misses and higher effective fetch bandwidth.

1.6. Store Buffer

JV32 does **not** implement a store buffer. Every store instruction issues its data memory write from the WB stage and the pipeline waits for the write to complete before the next instruction retires. With single-cycle TCM this is not a bottleneck because SRAM writes complete in one cycle. In a system with a slower memory bus or cache miss penalty a write buffer would improve performance by decoupling store retirement from bus transactions, allowing subsequent instructions to execute while the write is in flight.

[[Part II]] == Part II: Performance Analysis

1.7. Simulation Environment

All results are obtained by running the Verilator RTL simulator of the JV32 SoC with the following baseline parameters unless noted:

```
ARCH=rv32ima_zicsr  FAST_MUL=1  MUL_MC=0  FAST_DIV=1  FAST_SHIFT=1  BP_EN=1
```

The `rv32ima_zicsr` ISA selects RV32IMA with CSR instructions but **without** the C (compressed) extension; `rv32imac_zicsr` additionally enables 16-bit compressed instructions. The simulation clock is 80 MHz (as assumed by the benchmarks for score reporting).

CPI is measured directly from the RTL trace: total clock cycles divided by total retired instructions, both reported by the Verilator driver at end-of-simulation.

1.8. Maximum Performance Results

1.8.1. CoreMark

CoreMark v1.0 with 4 iterations (`-O3 -DITERATIONS=4 -DPERFORMANCE_RUN=1`).

Metric	Value
CoreMark score	374.32 iterations/s
CoreMark/MHz	**3.74**
Total cycles	1,139,449
Retired instructions	1,033,749
CPI	**1.102**
Conditional branches	196,892 (19.0 % of insns)
Taken branches	104,009 (52.8 % taken rate)
BP mispredictions	42,471 (21.6 % mispred rate, 78.4 % accuracy)
JAL (pre-decoded)	10,404 (0 % miss)
JALR	1,252

1.8.2. Dhrystone

Dhrystone 2.1 with 1,000 runs (`-O3 -DREG= -DNO_PROTOTYPES`).

Metric	Value
Dhrystones/s	250,000
DMIPS	142
DMIPS/MHz	**1.42**
Total cycles	461,512
Retired instructions	344,473
CPI	**1.340**
Conditional branches	58,195 (16.9 % of insns)
Taken branches	31,074 (53.4 % taken rate)

Metric	Value
BP mispredictions	5,108 (8.8 % mispred rate, 91.2 % accuracy)
JAL (pre-decoded)	10,644 (0 % miss)
JALR	9,585

1.9. Branch Predictor Impact Analysis

To isolate the contribution of the branch predictor, `BP_EN=0` is used (all branches default to predict-not-taken; JAL is no longer pre-decoded). All other parameters are unchanged.

1.9.1. Results Table

Configuration	CoreMark/MHz	CoreMark CPI	Cycles saved	DMIPS/MHz	Dhry CPI	Cycles saved
`rv32ima, BP_EN=1` (BTFNT + L0 + JAL)	3.74	1.102	—	1.42	1.340	—
`rv32ima, BP_EN=0` (predict-not-taken)	3.55	1.160	+59,014	1.42	1.439	+34,350

1.9.2. Analysis

CoreMark (`BP_EN=1` vs `BP_EN=0`):

- Without the predictor, every taken branch (52.8 % of 196,892 = 104,004) and every JAL (10,407) causes a 1-cycle redirect: 114,411 potential stall cycles from control flow alone.
- With the predictor, only mispredicted branches (42,471) and JALR (1,252) cause a redirect: ~43,723 stall cycles. JAL is always free (pre-decoded).
- Measured cycle reduction: 1,198,463 − 1,139,449 = **59,014 cycles** (4.9 % of total).
- CPI improvement: 1.160 → 1.102 = **5.3 %**.

Dhrystone (`BP_EN=1` vs `BP_EN=0`):

- Dhrystone's branch pattern is dominated by a small set of hot loops. The L0 last-branch entry learns each loop head quickly and reduces the misprediction rate from 53.4 % (BP=0) to only 8.8 % (BP=1).
- Without the predictor: 31,074 branch mispredictions + 10,644 JAL misses = 41,718 stall cycles.

With predictor: 5,108 mispredictions + 0 JAL misses = 5,108.

- Measured cycle reduction: 495,862 − 461,512 = **34,350 cycles** (6.9 % of total).
- CPI improvement: 1.439 → 1.340 = **6.9 %**.

The larger relative gain on Dhrystone reflects the loop-heavy control flow: the L0 dynamic entry captures the dominant taken-loop pattern while BTFNT alone would not. CoreMark's more varied branch mix means the L0 entry gets evicted more frequently, leaving BTFNT's static rule as the primary predictor.

1.9.3. Misprediction Rate Comparison

Benchmark	BP_EN=0 mispred rate	BP_EN=0 accuracy	BP_EN=1 mispred rate	BP_EN=1 accuracy
CoreMark	52.8 %	47.2 %	21.6 %	78.4 %
Dhrystone	53.4 %	46.6 %	8.8 %	91.2 %

With `BP_EN=0` the misprediction counter is incremented for every taken branch (since the hardware always predicts not-taken) and for every JAL (since JAL pre-decode is disabled). The 100 % JAL miss rate and ~53 % branch misprediction rate confirm the expected behavior.

1.10. Compressed Instruction (RVC) Impact Analysis

To assess the effect of the RVC decompressor, the same benchmarks are compiled and simulated with `rv32imac_zicsr` (compressed enabled) and `rv32ima_zicsr` (no compressed). `BP_EN=1` throughout.

1.10.1. Results Table

Configuration	CoreMark/MHz	CoreMark CPI	DMIPS/MHz	Dhry CPI
`rv32ima_zicsr` (no RVC)	3.74	**1.102**	1.42	**1.340**
`rv32imac_zicsr` (with RVC)	3.66	1.124	1.42	1.355

1.10.2. Analysis

CoreMark:

- With compressed instructions the compiler generates a slightly different code layout. Total retired instructions change marginally (1,033,749 → 1,035,673, +0.2 %) while total cycles increase more (1,139,449 → 1,163,737, +2.1 %).
- Net CPI increase: 1.102 → 1.124 (+2.0 %).
- The overhead comes from RVC alignment bubbles: whenever a 32-bit instruction straddles a 4-byte boundary the RVC expander holds the pipeline for one extra cycle while the second half-word arrives.

Dhrystone:

- CPI increase: 1.340 → 1.355 (+1.1 %). The effect is smaller because Dhrystone has a higher proportion of memory-access and call/return instructions which compress well and benefit from shorter code paths.

Simulation vs. real hardware trade-off:

In the TCM-based simulation there is no instruction cache, so the reduced code size from compressed encoding brings no bandwidth benefit. On real silicon with an instruction cache, compressed code reduces cache pressure, lowers the eviction rate, and decreases power consumption — typically outweighing the small alignment overhead. The simulation results therefore **understate** the true benefit of RVC in hardware.

1.11. CPI Statistics from Trace Analysis

The following analysis is derived from the `--rtl-trace` output (`build/rtl_trace.txt`) by parsing instruction retirement cycle numbers. Each trace line records `N:C PC (INSTR) ⋯` where *N* is the retirement index and *C* is the clock cycle. The gap between consecutive retirement cycles reveals pipeline stalls.

Configuration: `ARCH=rv32ima_zicsr FAST_MUL=1 MUL_MC=0 FAST_DIV=1 FAST_SHIFT=1 BP_EN=1 TRACE=1`.

1.11.1. CPI Decomposition

The total CPI overhead is decomposed into two independent contributors by combining trace data with the hardware branch predictor counters:

$$\mathbf{CPI = 1.0\ (ideal) + CPI_{redirect} + CPI_{mem\text{-}stall}}$$

CPI Component	CoreMark value	CoreMark share	Dhrystone value	Dhrystone share
Ideal (1 instruction/cycle)	1.000	90.7 %	1.000	74.6 %
Redirect stalls (branch mispred + JALR)	0.042	3.8 %	0.043	3.2 %
Memory stalls (load-use + alignment)	0.060	5.5 %	0.297	22.2 %
Total CPI	**1.102**	—	**1.340**	—

Redirect stall contribution is calculated as: $CPI_{redirect}$ = (mispredictions + JALR count) / retired instructions.

Memory stall contribution is the remainder: $CPI_{mem\text{-}stall} = CPI - 1.0 - CPI_{redirect}$.

1.11.2. CoreMark Instruction Mix

Configuration: `rv32ima_zicsr`, 4-iteration performance run.

Instruction Class	Count	Share
ALU (R-type, I-type, LUI, AUIPC)	512,447	49.6 %
Memory loads	214,719	20.8 %
Conditional branches	196,892	19.0 %
Memory stores	59,583	5.8 %
MUL / DIV	38,249	3.7 %
Jumps (JAL + JALR)	11,655	1.1 %
CSR operations	3	0.0 %
Total	**1,033,749**	—

1.11.3. Dhrystone Instruction Mix

Configuration: `rv32ima_zicsr`, 1,000 runs.

Instruction Class	Count	Share
ALU (R-type, I-type, LUI, AUIPC)	184,532	53.6 %
Memory loads	83,752	24.3 %

Instruction Class	Count	Share
Conditional branches	58,195	16.9 %
Memory stores	53,346	15.5 %
Jumps (JAL + JALR)	20,228	5.9 %
MUL / DIV	2,172	0.6 %
CSR operations	13	0.0 %
Total	**344,473**	—

ALU count is computed as total minus the explicitly categorised classes; some instructions fall into multiple categories (e.g. AUIPC is both ALU and PC-relative) so the sum slightly exceeds 100 %.

1.11.4. Stall Cycle Distribution

The stall distribution is obtained by counting the cycle gap between consecutive retired instructions. Gap = 1 means no stall cycle; gap = k means k−1 stall cycles between two consecutive retirements.

Table 1. CoreMark stall distribution (`rv32ima_zicsr`, BP_EN=1)

Stall cycles between retirements	Count	Share of retired insns	Stall cycles contributed
0 (gap=1, no stall)	958,445	92.7 %	0
1 (gap=2)	50,092	4.8 %	50,092
2 (gap=3)	22,484	2.2 %	44,968
3 (gap=4)	1,585	0.2 %	4,755
5 (gap=6)	1,143	0.1 %	5,715
Total stall cycles	—	—	**105,530 (9.3 % of all cycles)**

Table 2. Dhrystone stall distribution (`rv32ima_zicsr`, BP_EN=1)

Stall cycles between retirements	Count	Share of retired insns	Stall cycles contributed
0 (gap=1, no stall)	286,704	83.2 %	0

Stall cycles between retirements	Count	Share of retired insns	Stall cycles contributed
1 (gap=2)	11,413	3.3 %	11,413
2 (gap=3)	36,774	10.7 %	73,548
3 (gap=4)	8,001	2.3 %	24,003
5 (gap=6)	1,581	0.5 %	7,905
Total stall cycles	—	—	**116,869 (25.3 % of all cycles)**

1.11.5. Observations

CoreMark is a compute-intensive workload with a balanced instruction mix. With 90.7 % of instructions retiring without any stall cycle, the pipeline is highly efficient. The dominant stall source is single-cycle gaps (4.8 %) from a combination of load-use hazards (~6,369 instances) and the 42,471 branch mispredictions. The 2-cycle stalls (2.2 %) arise from dependent load-then-load address chains (pointer dereference of a freshly loaded pointer), where the second load must wait for both the first load's data and its own memory response.

Dhrystone has a very different profile. Only 83.2 % of instructions retire without stalling, and memory-related stalls dominate. The 10.7 % share of 2-cycle stall events is large: Dhrystone's string operations (`strcpy`, `strcmp`) generate dense load/store sequences with heavy pointer chasing — the loaded pointer is immediately used as the address of the next load, creating a load→address→load chain that incurs one stall per hop. With 24.3 % loads and 15.5 % stores (40 % combined memory-access rate versus 26.6 % for CoreMark), load-use stalls dominate to such an extent that $CPI_{mem\text{-}stall}$ = 0.297 vs 0.060 for CoreMark — a 5× difference. The branch predictor contribution is nearly identical in absolute terms (0.042 vs 0.043 CPI) because both workloads have similar branch counts relative to total instructions.

Key insight: the bottleneck for Dhrystone is **load-use latency**, not branch prediction. A deeper pipeline with a dedicated memory stage (4-stage IF→EX→MEM→WB) would eliminate the load-use stall entirely; the current 3-stage design accepts this stall as the price of a simpler, shorter pipeline with lower absolute clock-cycle count for non-memory-intensive code.

1.12. Summary Table

Configuration	CoreMark/MHz	CoreMark CPI	DMIPS/MHz	Dhry CPI
`rv32ima`, BP=1, FAST_MUL=1 MUL_MC=0, FAST_DIV=1, FAST_SHIFT=1 **(max perf)**	**3.74**	**1.102**	**1.42**	**1.340**
`rv32ima`, BP=0 **(no branch predictor)**	3.55	1.160	1.42	1.439
`rv32imac`, BP=1 **(with compressed)**	3.66	1.124	1.42	1.355

Feature	CoreMark gain	Dhrystone gain
Branch predictor (BTFNT + L0 + JAL pre-decode)	+5.3 % CPI	+6.9 % CPI
Uncompressed vs compressed (rv32ima vs rv32imac)	+2.0 % CPI (simulation only)	+1.1 % CPI (simulation only)